PhysAgent: Physics-Informed SDevice Code Generation via Ontology-Guided LLM

Shinbin Kim*
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
sinbin0701@postech.ac.kr

Giung Kim*
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
giung0408@postech.ac.kr

Jiwon Lee†
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
jiwonlee@postech.ac.kr

Dae-Hwan Kang†
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
daehwankang@postech.ac.kr

***Abstract*—As semiconductor scaling reaches its limits, Technology Computer-Aided Design (TCAD) simulation becomes essential for Design-Technology Co-Optimization (DTCO) [1]. However, the core challenge lies in configuring SDevice—the physics-and-solver specification stage of Synopsys Sentaurus TCAD [2]—rather than in device structure generation. We propose PhysAgent, which builds an ontology from the Sentaurus TCAD guide, augmented with literature-driven domain knowledge, to generate physically grounded SDevice files without relying on synthetic training data.**

***Keywords*—TCAD, ontology, physics-informed modeling, parameter file generation, domain knowledge, semiconductor materials**

I. Introduction

Recent studies have applied LLMs to automate TCAD code generation, primarily targeting structure files written for SDE (Sentaurus Structure Editor). These approaches rely on supervised fine-tuning (SFT) with synthetic datasets constructed by varying numerical parameters in a small number of baseline scripts [3]–[6]. However, this limits generalization to unseen device architectures or physical configurations. More critically, no prior work has systematically addressed physics model selection or parameter file generation for SDevice—the core of physically meaningful TCAD simulation. Even AgenticTCAD [6], which generates SDevice code, uses only 2–3 fixed physics configurations and omits parameter files entirely.

We propose PhysAgent, which builds an ontology from the Sentaurus TCAD guide. The ontology captures physical models, their governing equations, and the interdependencies among them, augmented with material-specific knowledge drawn from the literature. By providing this structured context to general-purpose LLMs, PhysAgent generates physically grounded SDevice files with explicit reasoning about model selection and parameter settings—without domain-specific fine-tuning.

II. Proposed Framework

PhysAgent combines a TCAD physics ontology and literature-driven domain knowledge to guide LLM-based SDevice file generation. The constructed ontology is publicly available as an interactive web platform [7].

*These authors contributed equally to this work.
†Corresponding authors.

A. TCAD Physics Ontology

An SDevice simulation takes a command file (`.cmd`) and a parameter file (`.par`) as input. The physics block in the `.cmd` file specifies the physical models activated for each material, while the `.par` file defines the parameters required by these models along with fundamental properties such as bandgap, permittivity, and the DOS effective masses of electrons and holes. Thus, the model selection in the `.cmd` file determines the parameter set that must be provided in the `.par` file. To consistently capture this `.cmd`–`.par` dependency, we structure the relevant parameters, TCAD syntax, and physical knowledge into an ontology grounded in the Poisson and transport equations, together with the governing equations of each physical model. Each parameter and each physical-model equation is represented as a node. Parameter nodes attach to the equation node of the model they belong to (membership), and equation nodes link to one another to capture inter-model couplings (coupling). Given a model selection, traversing these edges yields the required parameter set. A syntax layer then applies the SDevice syntax to the selected equations and nodes to generate the `.cmd` and `.par` files (Fig. 1).

B. Ontology-Guided Generation

Given a target device and material, PhysAgent accesses the ontology through an MCP (Model Context Protocol) interface (referred to as `ontology_mcp`) that exposes four tools. `ontology_overview` returns the node/edge map of the graph, serving as the entry point for grasping the overall structure. `ontology_get_chapter` identifies the governing equations associated with a selected node and derives the parameter set required by those equations. `ontology_get_syntax` provides the SDevice syntax template for each model, specifying keyword names, section layout, and required fields. `ontology_get_reference` returns material-specific parameter values together with per-parameter source citations drawn from a curated collection of papers, datasheets, and TCAD references [8], [9]. In our experiments, we employ Claude Sonnet, whose context window is limited to 200k tokens, whereas the full ontology measures ∼458k tokens and therefore cannot be injected directly into the prompt. The MCP-based selective retrieval scheme circumvents this limitation by allowing the LLM to query only the information required at each step, keeping

the per-query context well within the 200k-token budget. Moreover, each parameter in the generated `.par` inherits the citation returned by `ontology_get_reference`, thereby enabling parameter-level provenance (Fig. 3).

III. Experimental Validations of PhysAgent

We compare three knowledge-source conditions under identical model (Anthropic `claude-sonnet-4-6`) and prompt (*"I want to simulate a silicon transistor in Sentaurus at 300 K. Produce the Physics block of the cmd file and the corresponding .par file."*), each executed in 10 independent trials on a Si nMOSFET (`.cmd` Physics block + `.par` generation): (i) no external grounding (`no_source`), (ii) a raw-text dump of Sentaurus manual excerpts (`SDevice manual`), and (iii) the ontology accessed through the four tools of Chapter II.B (`ontology_mcp`, the PhysAgent configuration). Decks are evaluated by (a) successful loading of the generated deck by the `sdevice` engine (**pass/fail** per trial; failure types **K/V/M** detailed in Fig. 2) and (b) required-model mention rate — the fraction of 14 required model keywords (e.g., `OldSlotboom`, `PhuMob`, `MultiValley`) that appear as case-insensitive substrings in the combined generated `.par` + Physics-block text. `no_source` hallucinates nonexistent keywords (0/10 pass, 39% mention rate); `SDevice manual` contains all the required information, yet the LLM still produces incorrect syntax and fabricates parameter values from memory rather than the governing equations — mention rate rises to 54% but every trial still fails to load (0/10 pass) (Fig. 2), motivating a structured ontology over raw prose. Only `ontology_mcp` achieves both successful loading and highest mention rate (10/10 pass, 81%) via a staged workflow: `ontology_overview` maps the full structure (equation relations, physical meaning, parameter locations); `ontology_get_chapter` then selects, from the prompt's target material, the `.par` parameters and `.cmd` physical models grounded in the governing equations; `ontology_get_reference` supplies the matching literature values (Fig. 3); and `ontology_get_syntax` renders the final SDevice-compliant output. The **structure—not the volume—of context** sets the quality ceiling in LLM-driven TCAD deck generation.

IV. Conclusion

We presented PhysAgent, which generates physically grounded SDevice files by combining an ontology built from the Sentaurus TCAD guide with literature-driven domain knowledge, enabling general-purpose LLMs to support expert-level TCAD modeling without synthetic training data. On a Si nMOSFET benchmark, PhysAgent achieves 10/10 pass at 81% mention rate, versus 0/10 pass for both raw-text and no-context baselines.

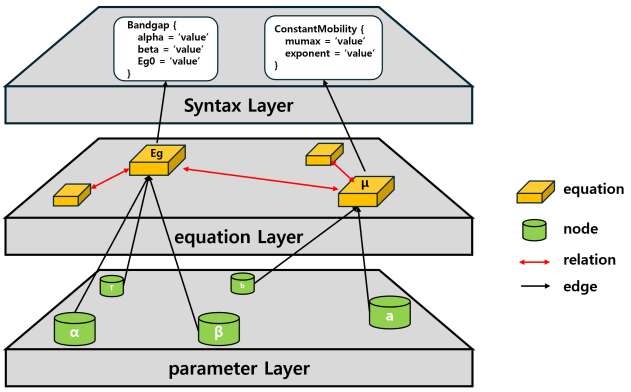

Fig. 1. TCAD physics ontology (678 equations, 1,631 parameters, 216 physical models, 30 literature references): node and edge types capturing inter-model dependencies.

Mode	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10	Dominant failure
(i) `no_source`	K	K	K	K	K	K	K	K	K	K	Hallucinated keyword
(ii) `SDevice manual`	V	M	V	V	V	M	M	M	V	M	Malformed syntax
(iii) PhysAgent (`ontology_mcp`)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	All pass

Fig. 2. Trial-by-trial outcomes (t1–t10, not aggregated) across three knowledge-source modes on a Si nMOSFET. **K** (hallucinated keyword): the LLM emits names drawn from foreign simulators—SPICE/BSIM (`epsilon_r`, `Bgap0`), Silvaco Atlas (`taun0`, `taup0`)—or fabricated symbols (`dEg_dT`, `Model=...`), none of which exist in SDevice syntax. **V** (invalid value): valid keyword but parameter value outside the physically admissible range or in an unsupported format. **M** (malformed syntax): correct keywords assembled into `.par` blocks with missing required fields or reordered sections that the SDevice parser rejects.

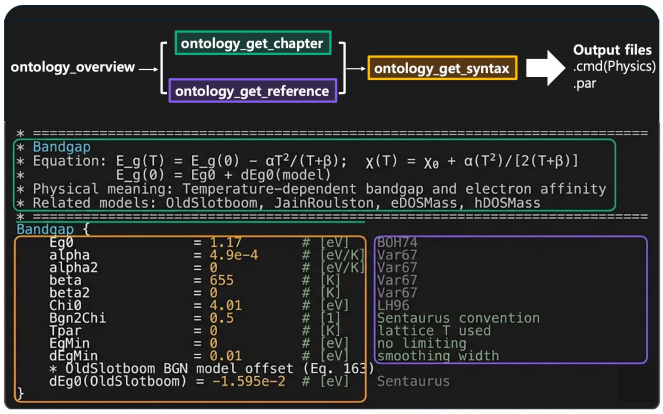

Fig. 3. PhysAgent framework and ontology-grounded `.par` file generation: `ontology_overview` maps the overall node/edge structure; `ontology_get_chapter` supplies the governing equations; `ontology_get_syntax` provides the SDevice syntax template; `ontology_get_reference` injects material-specific values with per-parameter citation tags. Example: Silicon Bandgap block.

References

[1] Z. Ji *et al.*, "Toward reliability- and variability-aware design-technology co-optimization in advanced nodes," *IEEE Trans. Electron Devices*, vol. 71, no. 1, pp. 138–150, Jan. 2024.

[2] Synopsys, "Sentaurus TCAD," 2025. [Online]. Available: https://www.synopsys.com/manufacturing/tcad.html

[3] L. M. L. Nguyen, A. Lu, and H. Y. Wong, "TCAD structure input file generation using large language model," in *Proc. SISPAD*, 2024, pp. 1–4.

[4] Z. Zhang, S. Nassif, and M. Tahoori, "ChatTCAD: Leveraging large language models for automated TCAD simulation file generation," in *Proc. ISVLSI*, 2025, pp. 1–6.

[5] J. Dou *et al.*, "MALTS: A multi-agent large language model-based layout-to-TCAD structure synthesizer," in *Proc. ISEDA*, 2025, pp. 795–800.

[6] G. Fan *et al.*, "AgenticTCAD: A LLM-based multi-agent frame-

work for automated TCAD code generation and device optimization," *arXiv:2512.23742*, 2026.

[7] S. Kim and G. Kim, "TCAD Physics Ontology," 2026. [Online]. Available: https://ontology-tcad.vercel.app/

[8] W. Bludau, A. Onton, and W. Heinke, "Temperature dependence of the band gap of silicon," *J. Appl. Phys.*, vol. 45, no. 4, pp. 1846–1848, Apr. 1974.

[9] Ioffe Inst., "NSM Archive: Silicon," [Online]. Available: https://www.ioffe.ru/SVA/NSM/Semicond/Si/. [Accessed: Apr. 23, 2026].